Programa de Articulação da Formação Profissional Média e Superior (AMS)

Disciplina:
Estruturas de Dados

Árvores

- Prof. Alexandre Ponce de Oliveira

alexandreponceo@gmail.com

Aula 43 e 44

Lins/SP

Árvores Binárias – Remoção de um nó

Situações para remover um nó:

Árvore vazia

- nada tem que ser feito

Árvore não for vazia

- compare o valor armazenado no nó raiz com x

Se maior, retire o elemento da subárvore à esquerda

Se menor, retire o elemento da subárvore à direita

Se for igual, retire a raiz da árvore

Árvores Binárias – Remoção de um nó

Existem três situações na remoção de um nó:

1ª situação: Nó sem filhos

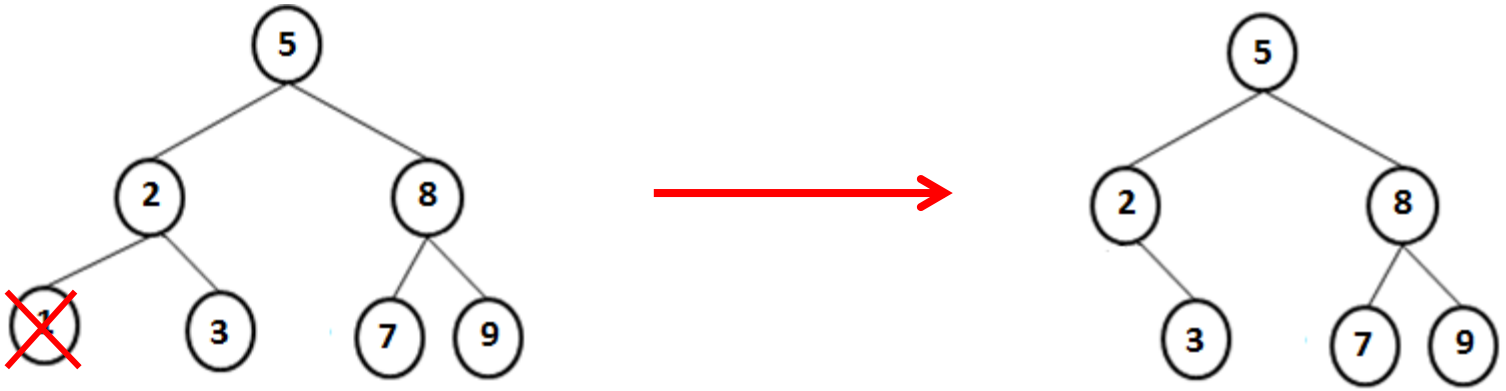
2ª situação: Nó com 1 filho (esquerdo ou direito)

3ª situação: Nó com 2 filhos

Árvores Binárias – Remoção de um nó

Situação 1: Nó sem filhos

Exemplo: Remover nó com valor 1



Árvores Binárias – Remoção de um nó

Situação 2: Nó com 1 filho (esquerdo ou direito)

Exemplo: Remover nó com valor 8



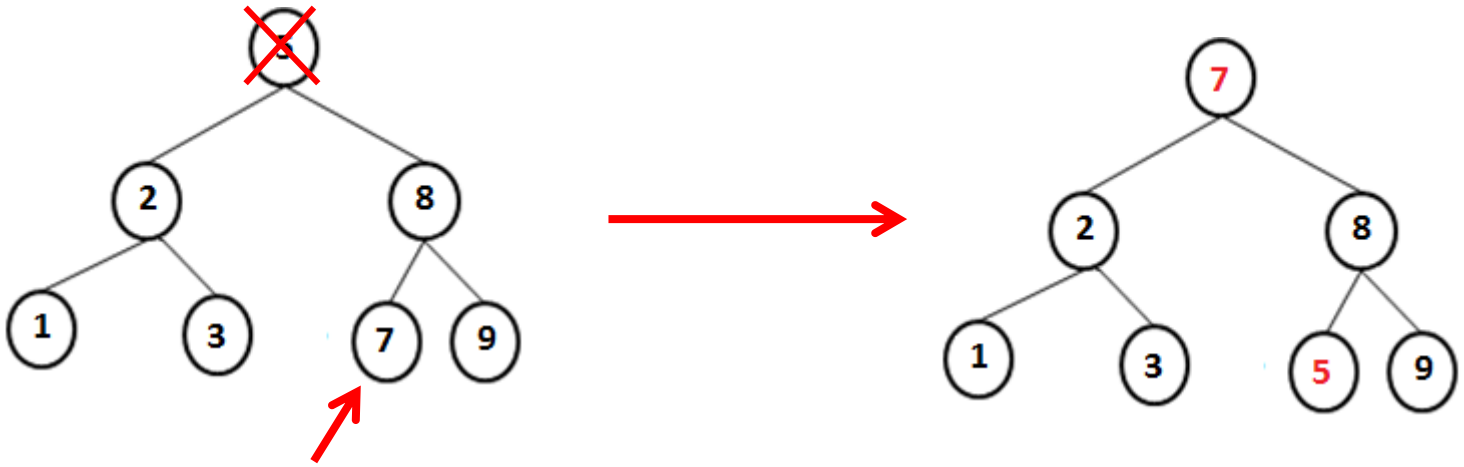
Árvores Binárias – Remoção de um nó

Situação 3: Nó com 2 filhos

- Encontrar o nó com valor que sucede a raiz na ordenação, no caso, o nó mais à esquerda da subárvore da direita.

- Trocar o valor do nó raiz com o troque o dado da raiz com o nó encontrado

Exemplo: Remover nó com valor 5

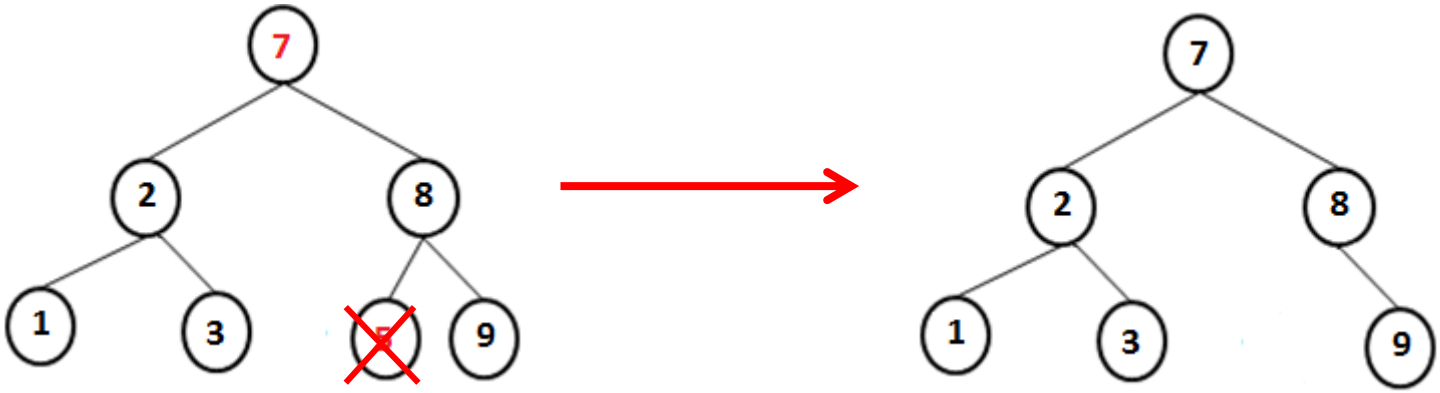


Árvores Binárias – Remoção de um nó

Situação 3: Nó com 2 filhos

- Na sequência, retire o nó encontrado da subárvore da direita

Exemplo: Remover nó com valor 5



Árvores Binárias

Função em C:

- arvore* exclui_no(arvore* a, int x);

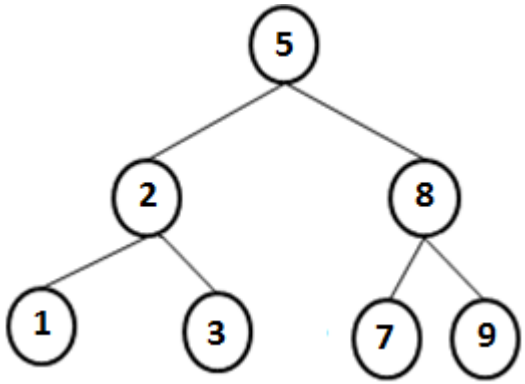
Código na função main():

```
printf("\nDigite um valor para excluir da arvore: ");
scanf("%d", &i);
a = exclui_no(a,i);
```

Árvores Binárias – Inserção Ordenada

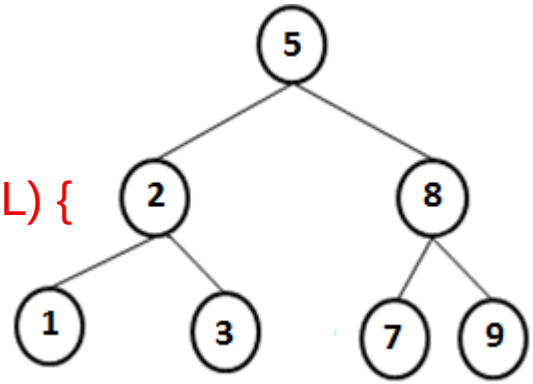
```
arvore* exclui_no(arvore* a, int x)
{
        if (a == NULL)
                return NULL;
        else if (a -> valor > x)
                a -> esq = exclui_no(a -> esq, x);
        else if (a -> valor < x)
                a -> dir = exclui_no(a -> dir, x);

        else { // encontrou o nó para remover
                .......
```



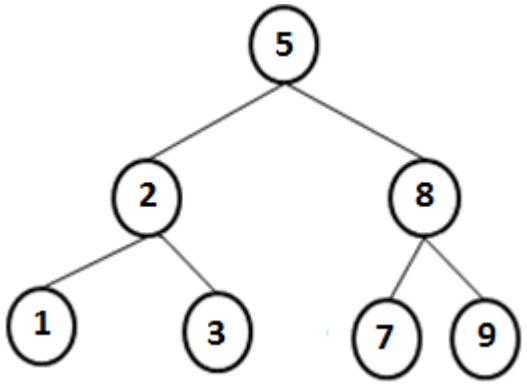
Árvores Binárias – Inserção Ordenada

```
.......
else { // encontrou o nó para remover
                    //nó sem filhos
                    if (a -> esq == NULL && a -> dir == NULL) {
                                printf("nó sem filhos");
                                free (a);
                                a = NULL;
                    } // nó com filho apenas na direita
                    else if (a -> esq == NULL) {
                                //printf("no com filho apenas na direita");
                                arvore* aux = a;
                                a = a -> dir;
                                free (aux);
                    }  // nó com filho apenas na esquerda
                    else if (a -> dir == NULL) {
                                //printf("no com filho apenas na esquerda");
                                arvore* aux = a;
                                a = a -> esq;
                                free (aux);
                    }
```



Árvores Binárias – Inserção Ordenada

```
        .......
        // nó com dois filhos
        else {
                arvore* aux = a -> dir;
                while (aux -> esq != NULL) {
                        aux = aux -> esq;
                }
                //printf("no com dois filhos");
                a -> valor = aux -> valor;
                aux -> valor = x;
                a -> dir = exclui_no(a -> dir,x);
        }
}
return a;
```



Árvores Binárias – Busca sem recursão

Função em C:

- arvore* busca_sem_recursao (arvore* Arvore, int x);

Código na função main():

```
printf("\nDigite um valor para pesquisar na arvore: ");
scanf("%d", &i);
a = busca_sem_recursao(a,i);
if (a == NULL)
        printf("\nValor nao encontrado!");
else
        printf("Encontrado!");
```

Árvores Binárias – Busca sem recursão

Código da função em C:

```
arvore* busca_sem_recursao (arvore* Arvore, int x)
{
  do {
        if (x < Arvore -> valor)
              Arvore = Arvore -> esq;
        else
              Arvore = Arvore -> dir;
  } while (Arvore != NULL && x != Arvore -> valor);
  return(Arvore);
}
```

Exercícios

Implemente funções para:

a) Encontrar o maior valor da árvore;

b) Encontrar o menor valor da árvore;

c) Contar o número de nós da árvore;

d) Somar todos os valores dos nós da árvore.